HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN MÔI TRƯỜNG BỂ NUÔI THỦY SẢN SỬ DỤNG ESP32, FREERTOS VÀ MQTT

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Mục tiêu thực hiện

Đề tài xây dựng hệ thống giám sát và điều khiển môi trường bể nuôi thủy sản sử dụng ESP32. Hệ thống thu thập dữ liệu nhiệt độ và khoảng cách đến mặt nước thông qua các cảm biến, sau đó xử lý và truyền dữ liệu lên giao diện Web bằng giao thức MQTT.

Chương trình sử dụng FreeRTOS để chia các công việc thành nhiều Task, giúp quá trình đọc cảm biến, xử lý điều khiển và truyền dữ liệu được thực hiện độc lập.

Các mục tiêu chính:

- Đọc khoảng cách đến mặt nước bằng cảm biến HC-SR04.
- Đo nhiệt độ nước bằng cảm biến DS18B20.
- Sử dụng FreeRTOS để quản lý các Task.
- Trao đổi dữ liệu giữa các Task thông qua Queue.
- Kết nối Wi-Fi và truyền dữ liệu bằng MQTT.
- Xây dựng giao diện Web để giám sát và điều khiển thiết bị.
- Hỗ trợ hai chế độ AUTO và MANUAL.

1.2. Chức năng của hệ thống

Hệ thống có các chức năng chính:

- Hiển thị khoảng cách và nhiệt độ đo được.
- Tự động điều khiển bơm và van xả theo các ngưỡng khoảng cách.
- Cho phép bật/tắt bơm và van xả thông qua giao diện Web.
- Hiển thị trạng thái hoạt động của thiết bị.
- Cập nhật thông số cảm biến bằng biểu đồ thời gian thực.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Giới thiệu về RTOS

RTOS (Real-Time Operating System) là hệ điều hành thời gian thực, được thiết kế để quản lý và thực hiện các công việc theo những yêu cầu về thời gian xác định.

Khác với chương trình tuần tự thông thường, RTOS cho phép chia chương trình thành nhiều tác vụ (Task). Mỗi Task có thể được thiết lập mức ưu tiên và được bộ lập lịch phân chia thời gian xử lý.

Ưu điểm của RTOS:

- Cho phép nhiều tác vụ hoạt động đồng thời.
- Quản lý và phân chia tài nguyên xử lý hiệu quả.
- Hỗ trợ thiết lập mức ưu tiên cho từng tác vụ.
- Đáp ứng tốt các ứng dụng yêu cầu xử lý theo thời gian thực.
- Có các cơ chế giao tiếp và đồng bộ giữa các Task.

RTOS thường được sử dụng trong các hệ thống nhúng, thiết bị IoT, hệ thống điều khiển tự động và thiết bị giám sát.

2.2. Giới thiệu FreeRTOS

FreeRTOS là hệ điều hành thời gian thực mã nguồn mở, được sử dụng phổ biến trong các hệ thống nhúng. FreeRTOS hỗ trợ nhiều dòng vi điều khiển, trong đó có ESP32.

FreeRTOS cung cấp các chức năng như tạo Task, quản lý mức ưu tiên, lập lịch thực thi, Queue, Semaphore và Mutex.

Ưu điểm của FreeRTOS:

- Có dung lượng nhỏ, phù hợp với vi điều khiển.
- Mã nguồn mở, dễ tích hợp và sử dụng.
- Hỗ trợ nhiều Task với mức ưu tiên khác nhau.
- Cung cấp các cơ chế trao đổi dữ liệu và đồng bộ.
- Phù hợp với các ứng dụng IoT cần xử lý nhiều công việc.

Nhược điểm của FreeRTOS:

- Cách lập trình phức tạp hơn chương trình tuần tự.
- Cần quản lý bộ nhớ và tài nguyên giữa các Task.
- Có thể xảy ra xung đột dữ liệu nếu thiết kế không hợp lý.
- Cần kiểm soát mức ưu tiên và thời gian thực thi để tránh ảnh hưởng đến hoạt động hệ thống.

Trong đề tài, FreeRTOS được triển khai trên ESP32 để quản lý ba Task chính: đọc dữ liệu cảm biến, xử lý điều khiển và truyền dữ liệu MQTT.

2.3. Cơ chế giao tiếp giữa các Task

Trong FreeRTOS, các Task có thể trao đổi dữ liệu và đồng bộ hoạt động thông qua nhiều cơ chế khác nhau. Hai cơ chế phổ biến là Semaphore và Queue.

a. Semaphore (Cờ hiệu)

Semaphore là cơ chế đồng bộ giữa các Task, sử dụng một cờ hiệu để thông báo sự kiện hoặc kiểm soát quyền truy cập tài nguyên.

Một số loại Semaphore thường gặp:

- Binary Semaphore: Có hai trạng thái, thường dùng để thông báo sự kiện giữa các Task.
- Counting Semaphore: Quản lý số lượng sự kiện hoặc tài nguyên.
- Mutex Semaphore: Bảo vệ tài nguyên dùng chung, hạn chế nhiều Task truy cập đồng thời không phù hợp.

Ưu điểm:

- Giúp đồng bộ hoạt động giữa các Task.
- Hạn chế xung đột khi truy cập tài nguyên dùng chung.
- Phù hợp với các hệ thống có nhiều tác vụ cần phối hợp.

Ứng dụng: Thông báo khi có dữ liệu mới, đồng bộ quá trình xử lý hoặc bảo vệ tài nguyên dùng chung.

b. Queue (Hàng đợi)

Queue là cơ chế truyền dữ liệu giữa các Task theo nguyên tắc hàng đợi. Một Task gửi dữ liệu vào Queue, Task khác nhận dữ liệu từ Queue để xử lý.

Ưu điểm:

- Cho phép truyền nhiều loại dữ liệu giữa các Task.
- Các Task không cần truy cập trực tiếp vào biến của nhau.
- Có thể thiết lập kích thước hàng đợi và thời gian chờ.
- Hỗ trợ đồng bộ quá trình gửi và nhận dữ liệu.

Ứng dụng: Truyền dữ liệu cảm biến, gửi lệnh điều khiển và trao đổi trạng thái giữa các Task.

Trong đề tài, Queue được sử dụng làm cơ chế giao tiếp chính giữa các Task. Ngoài ra, chương trình sử dụng Critical Section để bảo vệ các biến điều khiển dùng chung.

2.4. Giao thức MQTT

MQTT là giao thức truyền thông nhẹ, phù hợp với các hệ thống IoT. Giao thức hoạt động theo mô hình Publisher/Subscriber thông qua một máy chủ trung gian gọi là MQTT Broker.

- Publisher: Thiết bị gửi dữ liệu lên Broker.
- Subscriber: Thiết bị đăng ký nhận dữ liệu từ Broker.
- Broker: Tiếp nhận và chuyển tiếp dữ liệu giữa các thiết bị.

Trong đề tài, ESP32 kết nối với MQTT Broker HiveMQ thông qua Wi-Fi. ESP32 gửi dữ liệu cảm biến lên Broker và nhận các lệnh điều khiển từ giao diện Web.

CHƯƠNG 3. THIẾT KẾ VÀ KẾT NỐI HỆ THỐNG

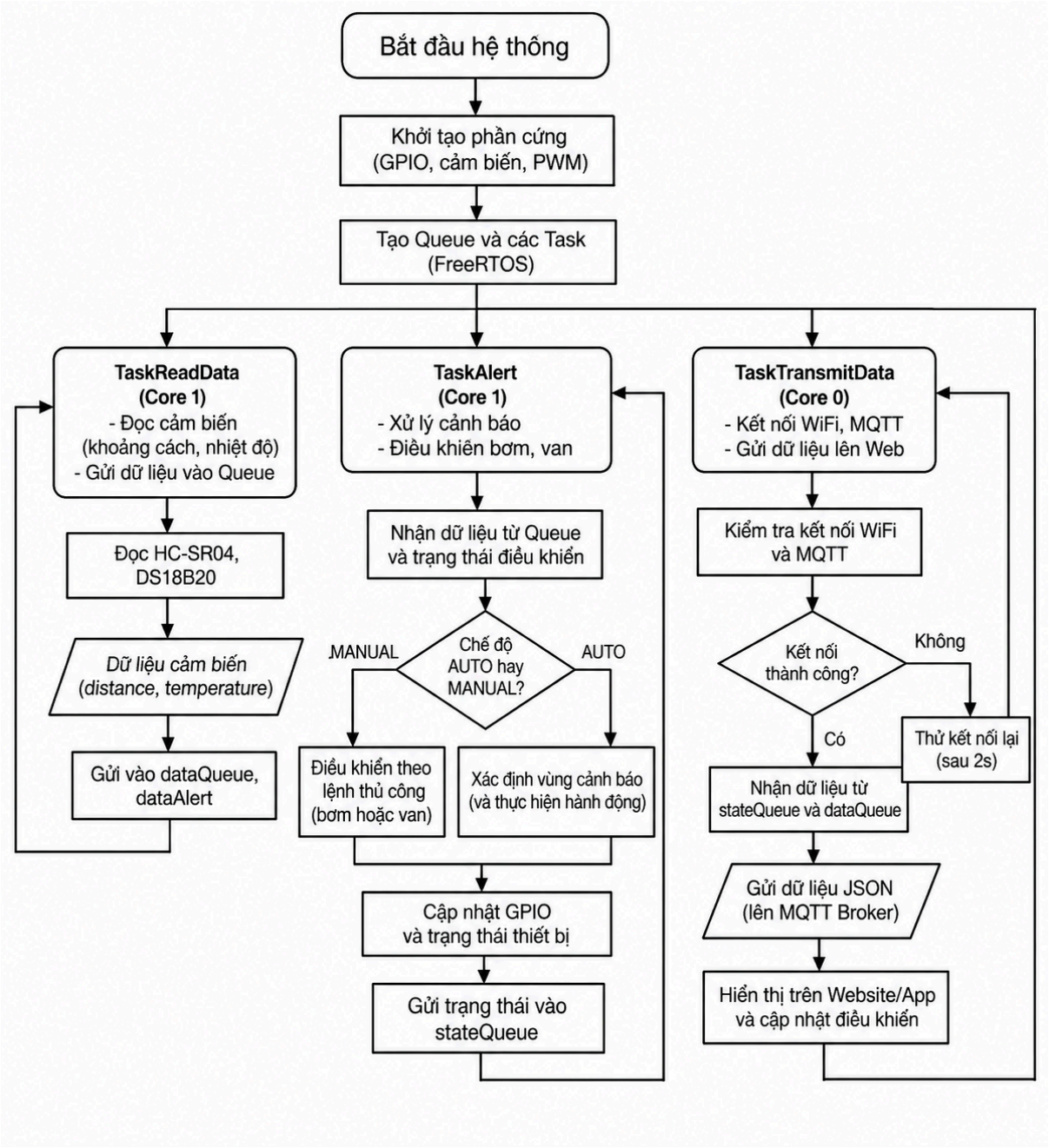
3.1. Sơ đồ khối hệ thống

Hệ thống gồm năm khối chính:

1. Khối cảm biến: HC-SR04 đo khoảng cách đến mặt nước, DS18B20 đo nhiệt độ nước.
2. Khối xử lý: ESP32 tiếp nhận dữ liệu và thực hiện các tác vụ FreeRTOS.
3. Khối truyền thông: Kết nối Wi-Fi và trao đổi dữ liệu thông qua MQTT Broker.
4. Khối giao diện: Website hiển thị dữ liệu cảm biến và nhận lệnh điều khiển.
5. Khối thiết bị chấp hành: Bơm và van xả, được mô phỏng bằng hai LED.

Nguyên lý hoạt động của hệ thống:

- Cảm biến gửi dữ liệu về ESP32.
- ESP32 xử lý dữ liệu và xác định trạng thái môi trường.
- Dữ liệu được gửi lên MQTT Broker.
- Website nhận dữ liệu và hiển thị các thông số.
- Người dùng có thể gửi lệnh điều khiển từ Website.
- ESP32 tiếp nhận lệnh và điều khiển thiết bị tương ứng.



Hình 3.1. Lưu đồ thuật toán

3.2. Các linh kiện sử dụng

STT	Linh kiện	Chức năng
1	ESP32 DevKit V1	Vi điều khiển trung tâm

2	HC-SR04	Đo khoảng cách đến mặt nước
3	DS18B20	Đo nhiệt độ nước
4	LED đỏ	Mô phỏng máy bơm
5	LED xanh	Mô phỏng van xả
6	Điện trở 220 Ω	Hạn dòng cho LED
7	Điện trở 4,7 kΩ	Điện trở kéo lên cho DS18B20

3.3. Sơ đồ kết nối và phân công chân GPIO

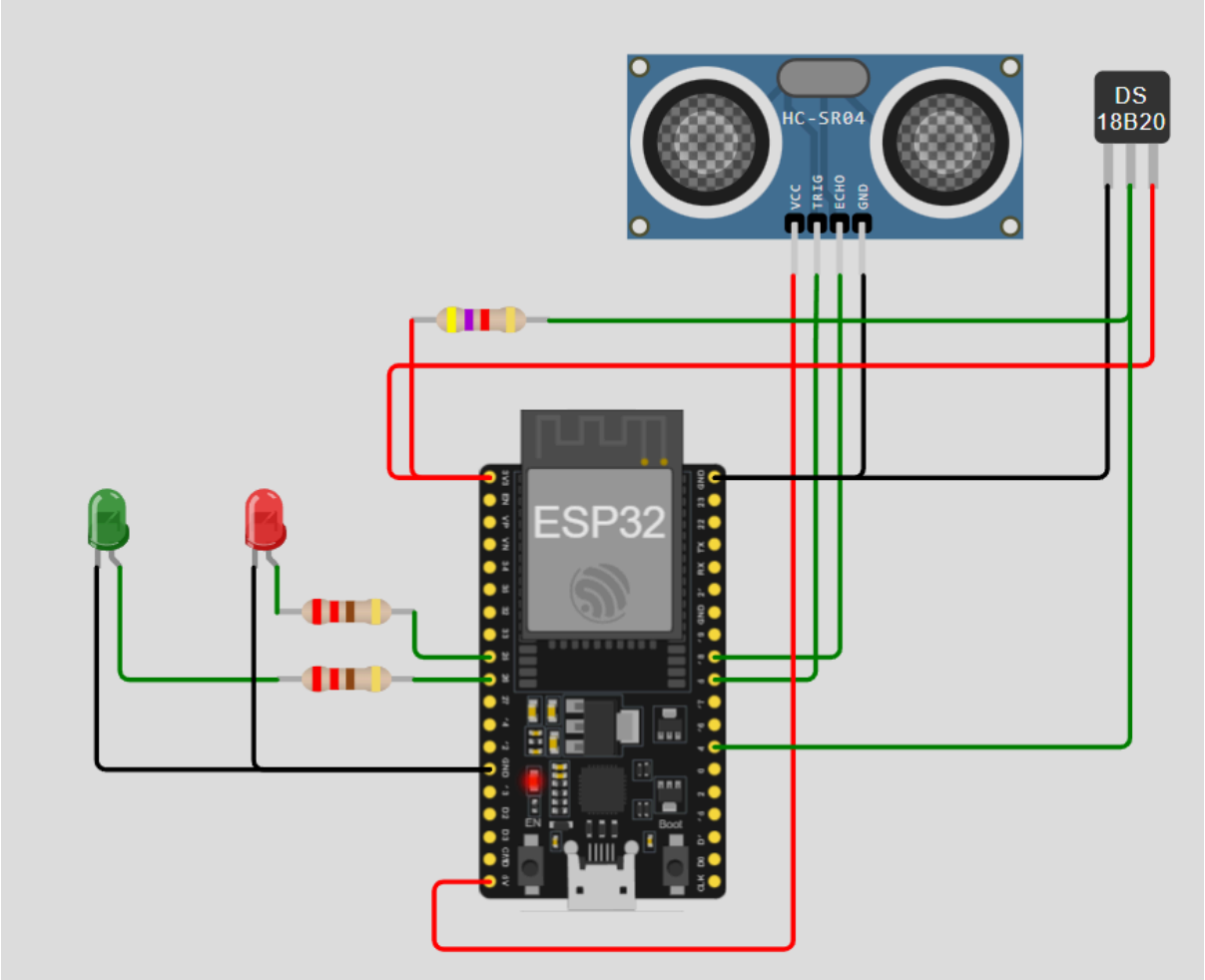
Các linh kiện được kết nối với ESP32 theo bảng sau:

Bảng 3.1. Bảng kết nối chân GPIO

Thiết bị	**Chân thiết bị**	**Chân ESP32**
HC-SR04	VCC	VIN/5V
HC-SR04	TRIG	GPIO 5
HC-SR04	ECHO	GPIO 18
HC-SR04	GND	GND

DS18B20	VCC	3V3
DS18B20	DATA	GPIO 4
DS18B20	GND	GND
LED đỏ	Anode qua điện trở	GPIO 25
LED xanh	Anode qua điện trở	GPIO 26

Chân DATA của DS18B20 được nối với điện trở kéo lên 4,7 kΩ và nguồn 3,3 V. Các chân cathode của LED được nối với GND.



Hình 3.2 – Sơ đồ mạch kết nối trên Wokwi tại đây

3.4. Nguyên lý điều khiển thiết bị

Khoảng cách được đo là khoảng cách từ cảm biến HC-SR04 đến mặt nước. Khi khoảng cách giảm, mặt nước tiến gần đến cảm biến hơn.

Hệ thống thiết lập các ngưỡng điều khiển như sau:

Bảng 3.2. Điều kiện điều khiển tự động

Khoảng cách	Thiết bị hoạt động	Thời gian
1 m đến dưới 1,5 m	Bơm 50%	10 giây
1,5 m đến 2 m	Bơm 100%	20 giây
Nhỏ hơn 0,5 m	Van xả mở	10 giây
Các khoảng cách còn lại	Tắt thiết bị	Không kích hoạt

ESP32 sử dụng PWM để điều chỉnh mức hoạt động của bơm. Trong mô phỏng, bơm và van xả được thay thế bằng LED để thể hiện trạng thái bật/tắt.

CHƯƠNG 4. TRIỂN KHAI FREERTOS TRÊN ESP32

4.1. Phân chia các Task

Chương trình được chia thành ba Task chính, mỗi Task đảm nhiệm một chức năng riêng.

Bảng 4.1. Bảng phân công các Task

Tên Task	Core	Priority	Stack	Chu kỳ/Hoạt động	Chức năng
TaskReadData	1	2	4096	2 giây	Đọc cảm biến, hiển thị Serial và gửi dữ liệu vào Queue
TaskAlert	1	2	4096	50 ms	Kiểm tra ngưỡng, xử lý điều khiển thiết bị
TaskTransmitData	0	1	8192	Liên tục	Kết nối Wi-Fi, MQTT, gửi dữ liệu và nhận lệnh

Trong đó:

- Core 0 đảm nhiệm việc truyền nhận dữ liệu MQTT.
- Core 1 đảm nhiệm việc đọc cảm biến và xử lý điều khiển.
- TaskReadData và TaskAlert có mức ưu tiên 2.
- TaskTransmitData có mức ưu tiên 1.

Việc phân chia Task giúp các công việc được tổ chức độc lập, tránh tập trung toàn bộ chức năng trong một vòng lặp duy nhất.

4.2. Cơ chế trao đổi dữ liệu giữa các Task

Chương trình sử dụng ba Queue:

Bảng 4.2. Các Queue trong hệ thống

Tên Queue	Kích thước	Kiểu dữ liệu	Chức năng
dataQueue	5	SensorData	Truyền khoảng cách và nhiệt độ đến Task MQTT

dataAlert	1	float	Truyền khoảng cách mới nhất đến Task điều khiển
stateQueue	1	DeviceState	Truyền trạng thái thiết bị đến Task MQTT

Cấu trúc dữ liệu cảm biến:

```
None
struct SensorData {
  float distance;
  float temperature;
};
```

Cấu trúc lưu trạng thái thiết bị:

```
None
struct DeviceState {
  bool autoMode;
  bool pumpOn;
  int pumpDuty;
  bool valveOpen;
};
```

Trong chương trình:

- xQueueSend() được sử dụng để gửi dữ liệu cảm biến vào hàng đợi.
- xQueueReceive() được sử dụng để nhận dữ liệu từ hàng đợi.
- xQueueOverwrite() được sử dụng với Queue có kích thước 1 để cập nhật dữ liệu hoặc trạng thái mới nhất.

Đối với dataAlert và stateQueue, chương trình chỉ cần giữ lại giá trị mới nhất nên sử dụng Queue kích thước 1.

4.3. Cơ chế bảo vệ dữ liệu dùng chung

Các biến autoMode, manualPump và manualValve được sử dụng để lưu chế độ hoạt động và lệnh điều khiển thiết bị.

Để hạn chế xung đột khi nhiều ngữ cảnh truy cập các biến này, chương trình sử dụng Critical Section:

```
None
portENTER_CRITICAL(&commandMux);

// Đọc hoặc cập nhật các biến điều khiển

portEXIT_CRITICAL(&commandMux);
```

Cơ chế này đảm bảo đoạn lệnh truy cập dữ liệu dùng chung được thực hiện trong vùng tới hạn, hạn chế việc thay đổi dữ liệu không mong muốn.

4.4. Nguyên lý hoạt động của các Task

TaskReadData:

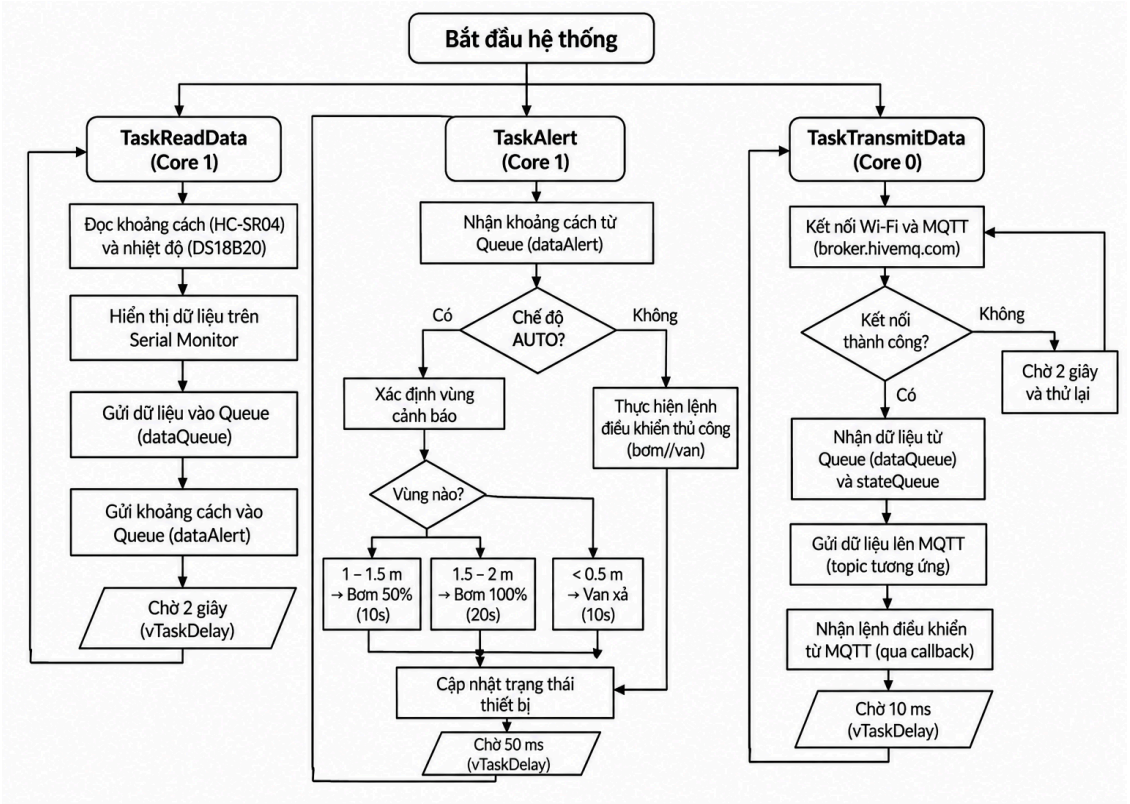
- Đọc khoảng cách từ HC-SR04.
- Đọc nhiệt độ từ DS18B20.
- Hiển thị kết quả trên Serial Monitor.
- Gửi dữ liệu vào dataQueue.
- Gửi khoảng cách mới nhất vào dataAlert.
- Chờ 2 giây trước khi đọc lần tiếp theo.

TaskAlert:

- Nhận khoảng cách từ dataAlert.
- Xác định vùng điều khiển.
- Kiểm tra chế độ AUTO hoặc MANUAL.
- Điều khiển bơm bằng PWM và van xả bằng GPIO.
- Cập nhật trạng thái vào stateQueue khi có thay đổi.

TaskTransmitData:

- Kết nối Wi-Fi.
- Kết nối MQTT Broker HiveMQ.
- Đăng ký nhận các topic điều khiển.
- Nhận dữ liệu từ dataQueue và gửi lên MQTT.
- Nhận trạng thái từ stateQueue và cập nhật lên Broker.
- Tiếp nhận các lệnh điều khiển từ Website.



Hình 4.1 – Sơ đồ luồng hoạt động của ba Task

CHƯƠNG 5. XÂY DỰNG GIAO DIỆN WEBSITE

5.1. Công nghệ sử dụng

Website được xây dựng bằng các công nghệ:

Công nghệ	Chức năng
HTML	Xây dựng cấu trúc giao diện
CSS	Thiết kế bố cục và màu sắc
JavaScript	Xử lý dữ liệu và điều khiển

MQTT.js	Kết nối và trao đổi dữ liệu MQTT
Chart.js	Hiển thị biểu đồ dữ liệu

Website kết nối với MQTT Broker thông qua MQTT over WebSocket để trao đổi dữ liệu với ESP32.

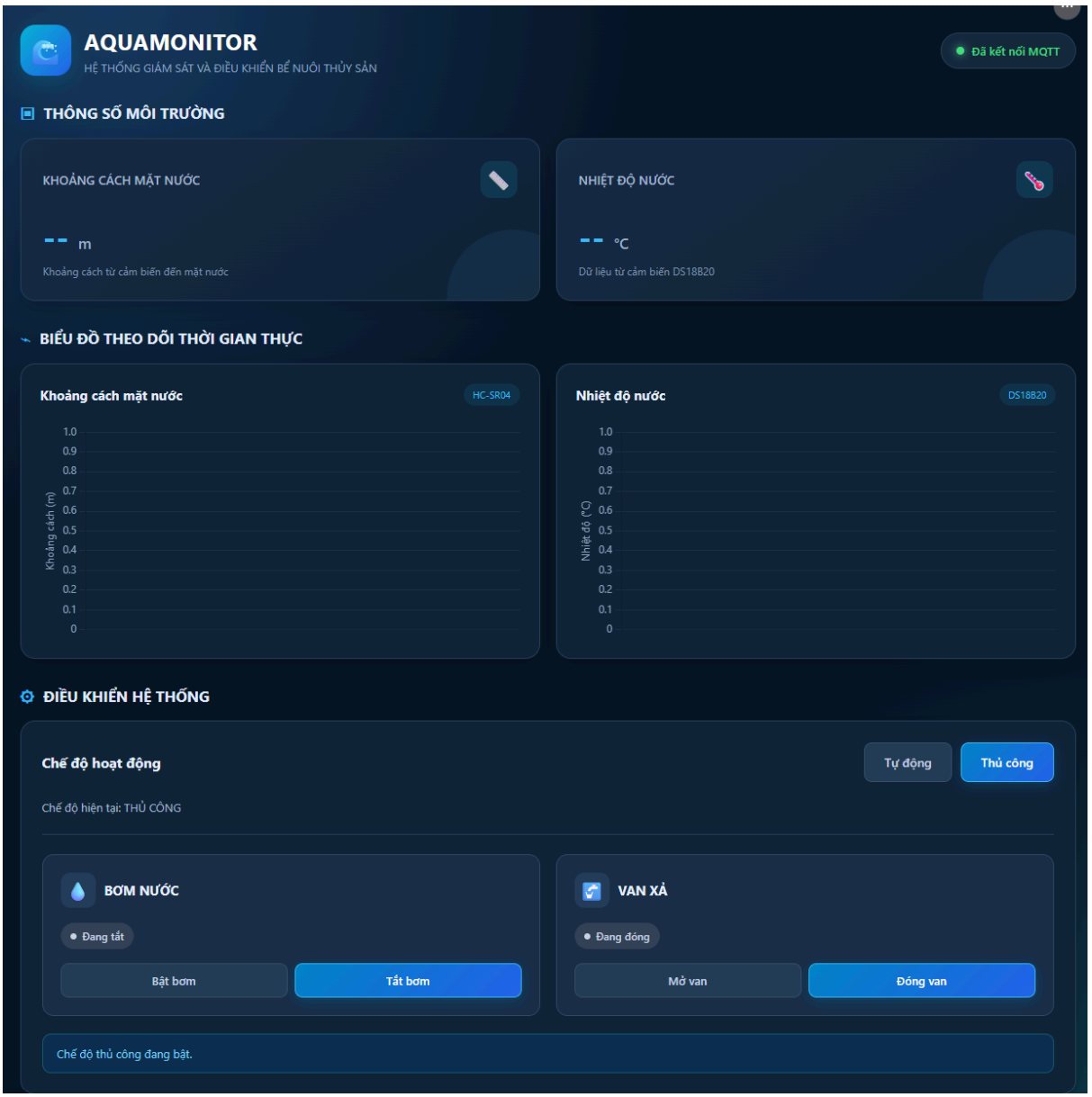
5.2. Giao diện giám sát

Giao diện chính hiển thị hai thông số:

- Khoảng cách từ cảm biến đến mặt nước.
- Nhiệt độ nước.

Ngoài các thẻ thông số, Website còn có hai biểu đồ đường để theo dõi sự thay đổi của khoảng cách và nhiệt độ theo thời gian thực.

Các giá trị mới được cập nhật khi Website nhận được bản tin MQTT từ ESP32. Mỗi biểu đồ lưu tối đa 20 điểm dữ liệu gần nhất để đảm bảo giao diện dễ quan sát.



Hình 5.1 – Giao diện Website hiển thị thông số và biểu đồ

5.3. Giao diện điều khiển

Website cung cấp hai chế độ hoạt động:

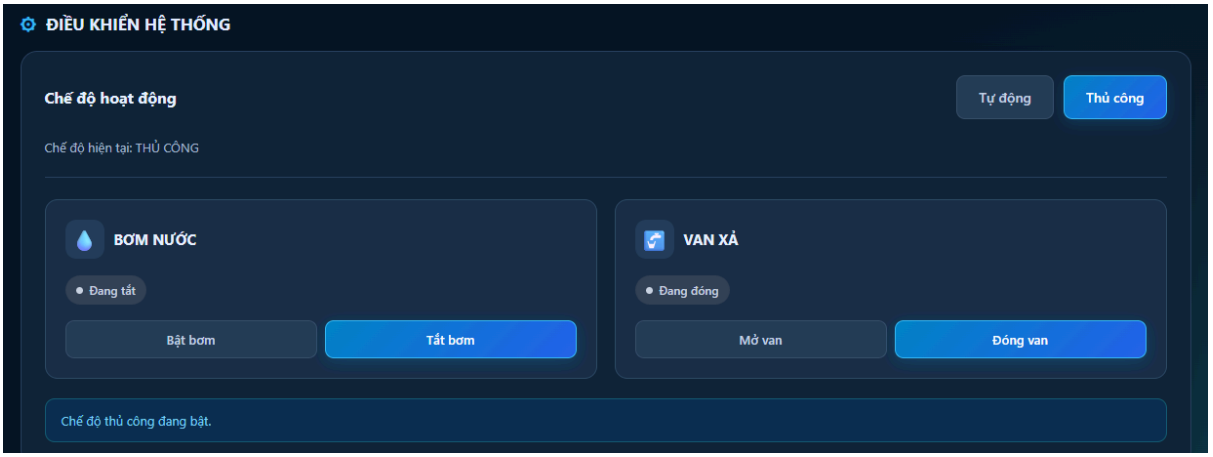
Chế độ AUTO:

ESP32 tự động điều khiển bơm và van xả dựa trên các ngưỡng khoảng cách đã thiết lập. Các nút điều khiển thủ công trên Website được vô hiệu hóa.

Chế độ MANUAL:

Người dùng có thể bật hoặc tắt bơm và van xả trực tiếp trên giao diện. Khi nhận lệnh, ESP32 cập nhật trạng thái thiết bị và gửi kết quả trở lại Website.

Hệ thống cũng hiển thị trạng thái bơm, van xả, chế độ hoạt động và tình trạng kết nối MQTT.



Hình 5.2 – Giao diện điều khiển ở chế độ MANUAL

CHƯƠNG 6. XÂY DỰNG CHƯƠNG TRÌNH VÀ GIẢI THÍCH CODE

6.1. Khai báo thư viện và cấu hình chân kết nối

Chương trình sử dụng các thư viện:

```
None
#include <WiFi.h>
#include <PubSubClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <math.h>
#include <string.h>
```

Trong đó:

- WiFi.h: Kết nối ESP32 với mạng Wi-Fi.
- PubSubClient.h: Thực hiện giao tiếp MQTT.
- OneWire.h: Hỗ trợ giao tiếp OneWire.
- DallasTemperature.h: Đọc dữ liệu từ cảm biến DS18B20.
- math.h: Kiểm tra giá trị số bằng hàm isnan().
- string.h: Xử lý chuỗi ký tự.

Các chân GPIO được khai báo:

```
None
#define TRIG_PIN  5
#define ECHO_PIN  18
#define TEMP_PIN  4
#define PUMP_PIN  25
#define VALVE_PIN 26
```

6.2. Kết nối Wi-Fi và MQTT

ESP32 sử dụng mạng Wi-Fi Wokwi-GUEST để kết nối Internet trong môi trường mô phỏng.

MQTT Broker được sử dụng là HiveMQ với địa chỉ broker.hivemq.com và cổng 1883.

```
None
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
```

Sau khi kết nối Wi-Fi thành công, ESP32 thực hiện kết nối MQTT. Nếu kết nối bị gián đoạn, chương trình sẽ thực hiện kết nối lại.

Khi kết nối MQTT thành công, ESP32 đăng ký ba topic điều khiển:

- Topic nhận lệnh chuyển chế độ AUTO/MANUAL.
- Topic nhận lệnh bật/tắt bơm.
- Topic nhận lệnh bật/tắt van xả.

6.3. Đọc dữ liệu cảm biến

Hàm readDistance() thực hiện đo khoảng cách bằng HC-SR04.

ESP32 phát xung TRIG trong 10 micro giây, sau đó đo thời gian tín hiệu phản hồi tại chân ECHO.

Công thức tính khoảng cách:

$$d = \frac{t \times 0{,}0343}{2}$$

Trong đó:

- d: Khoảng cách tính bằng cm.
- t: Thời gian truyền sóng tính bằng micro giây.
- 0,0343: Vận tốc truyền âm trong không khí, tính bằng cm/µs.

Kết quả sau đó được đổi từ cm sang m để phục vụ xử lý điều khiển.

Đối với DS18B20, chương trình sử dụng:

```
None
tempSensor.requestTemperatures();
data.temperature = tempSensor.getTempCByIndex(0);
```

Hai giá trị khoảng cách và nhiệt độ được lưu vào cấu trúc SensorData để truyền sang các Task khác.

6.4. Xử lý điều khiển tự động

Trong TaskAlert, chương trình xác định vùng điều khiển dựa trên khoảng cách nhận được.

```
None
if (distance >= 1.0 && distance < 1.5) {
   currentZone = 1;
}
else if (distance >= 1.5 && distance <= 2.0) {
   currentZone = 2;
}
else if (distance >= 0 && distance < 0.5) {
   currentZone = 3;
}
```

Sau khi xác định vùng, chương trình lựa chọn hành động tương ứng:

- Vùng 1: Bơm 50% trong 10 giây.
- Vùng 2: Bơm 100% trong 20 giây.
- Vùng 3: Mở van xả trong 10 giây.

Mức PWM được thiết lập bằng giá trị 128 tương ứng khoảng 50% và 255 tương ứng 100% với độ phân giải 8 bit.

Thời gian kết thúc hành động được xác định bằng millis(), giúp chương trình kiểm tra thời gian mà không cần sử dụng hàm delay() dài để chờ thiết bị hoạt động.

6.5. Xử lý chế độ điều khiển thủ công

Khi Website gửi lệnh MANUAL, ESP32 chuyển sang chế độ điều khiển thủ công.

Hàm mqttCallback() tiếp nhận bản tin từ MQTT Broker và kiểm tra topic để xác định loại lệnh.

Nếu nhận lệnh bật bơm, chương trình đặt manualPump = true và tắt yêu cầu mở van. Nếu nhận lệnh mở van, chương trình tắt yêu cầu bật bơm.

Nhờ đó, bơm và van xả không được yêu cầu hoạt động đồng thời.

Khi chuyển lại chế độ AUTO, các lệnh thủ công được xóa và hệ thống tiếp tục kiểm tra dữ liệu cảm biến để thực hiện điều khiển tự động.

6.6. Đóng gói và truyền dữ liệu MQTT

Dữ liệu cảm biến được đóng gói dưới dạng JSON:

```
None
snprintf(
    payload,
    sizeof(payload),
    "{\"distance\":%s,\"temperature\":%s}",
    distanceText,
    temperatureText
);
```

Ví dụ bản tin gửi lên MQTT:

```
None
{
  "distance": 1.25,
  "temperature": 28.50
}
```

ESP32 gửi bản tin này lên topic cảm biến. Website đăng ký nhận topic tương ứng và sử dụng JavaScript để phân tích dữ liệu JSON, cập nhật các thẻ thông số và biểu đồ.

Trạng thái thiết bị được đóng gói riêng, bao gồm chế độ hoạt động, trạng thái bơm, mức PWM và trạng thái van xả.

6.7. Khởi tạo hệ thống

Trong hàm setup(), chương trình thực hiện:

- Khởi tạo Serial Monitor.
- Cấu hình các chân GPIO.
- Khởi tạo PWM cho bơm.
- Khởi tạo cảm biến DS18B20.
- Tạo ba Queue.
- Kiểm tra kết quả tạo Queue.
- Tạo ba Task và phân công vào hai lõi xử lý của ESP32.

Sau khi khởi tạo, các công việc chính được thực hiện bởi FreeRTOS. Hàm loop() chỉ duy trì chương trình bằng vTaskDelay().

```
// Tao Queue du lieu cam bien
dataQueue = xQueueCreate(
  5,
  sizeof(SensorData)
);

// Tao Queue khoang cach canh bao
dataAlert = xQueueCreate(
  1,
  sizeof(float)
);

// Tao Queue trang thai thiet bi
stateQueue = xQueueCreate(
  1,
  sizeof(DeviceState)
);

// Kiem tra Queue
if (dataQueue == NULL ||
    dataAlert == NULL ||
    stateQueue == NULL) {

  Serial.println("Loi tao Queue!");

  while (true) {
    delay(1000);
  }
}
```

```
// Tao Task doc du lieu - Core 1
xTaskCreatePinnedToCore(
  TaskReadData,
  "TaskReadData",
  4096,
  NULL,
  2,
  NULL,
  1
);

// Tao Task canh bao va dieu khien - Core 1
xTaskCreatePinnedToCore(
  TaskAlert,
  "TaskAlert",
  4096,
  NULL,
  2,
  NULL,
  1
);

// Tao Task truyen du lieu MQTT - Core 0
xTaskCreatePinnedToCore(
  TaskTransmitData,
  "TaskTransmitData",
  8192,
  NULL,
  1,
  NULL,
  0
);

Serial.println("He thong da khoi tao!");
```

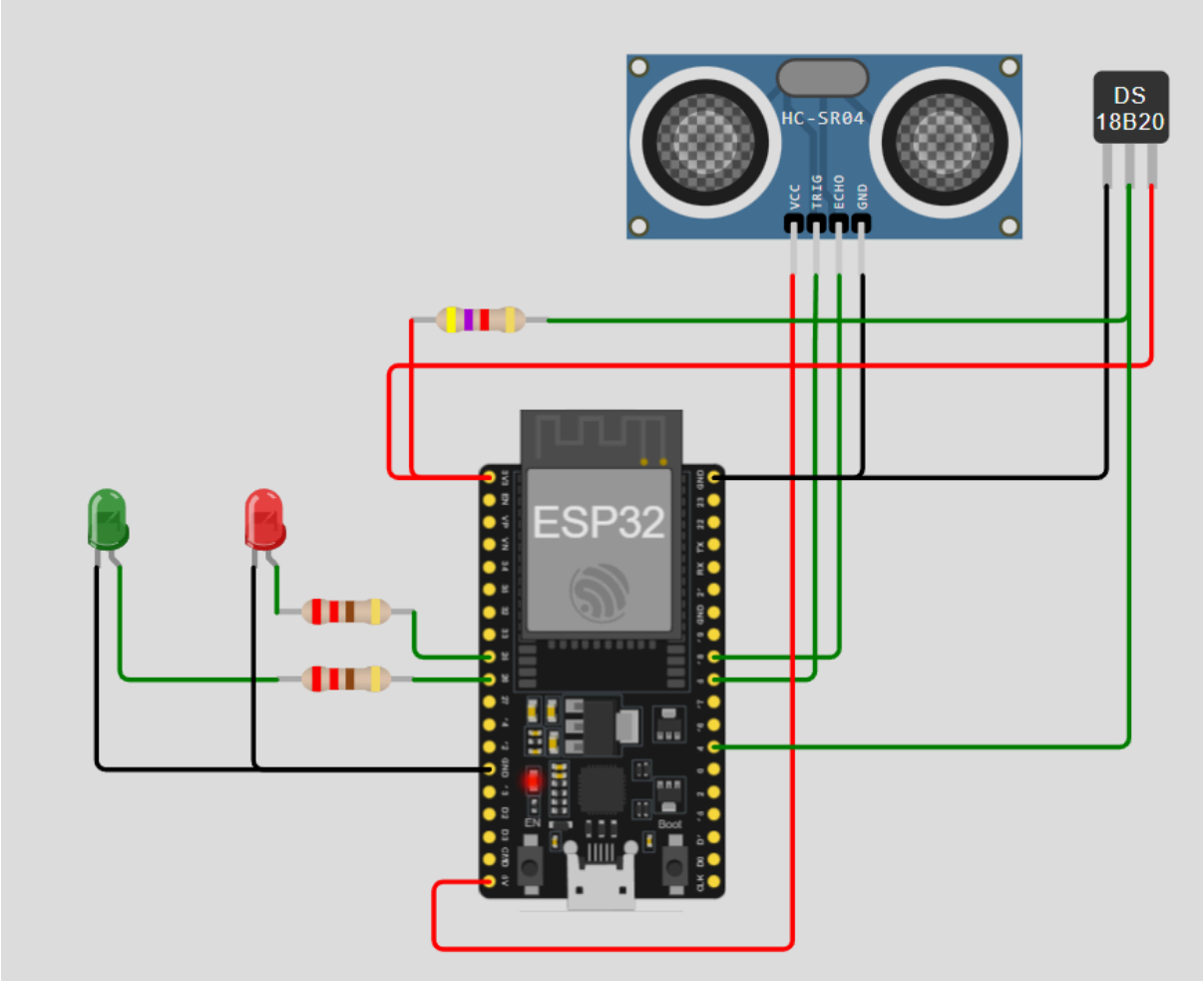
Hình 6.1 – Một phần mã nguồn khởi tạo Queue và các Task

CHƯƠNG 7. KẾT QUẢ THỰC HIỆN VÀ ĐÁNH GIÁ

7.1. Kết quả mô phỏng trên Wokwi

Hệ thống được xây dựng trên Wokwi với ESP32, cảm biến HC-SR04, DS18B20 và hai LED mô phỏng thiết bị chấp hành.

Sau khi khởi chạy, ESP32 có thể đọc dữ liệu cảm biến, thực hiện các Task FreeRTOS và kết nối MQTT Broker.



Hình 7.1 – Toàn bộ mạch mô phỏng trên Wokwi

7.2. Kết quả đọc dữ liệu cảm biến

Dữ liệu khoảng cách và nhiệt độ được hiển thị trên Serial Monitor theo chu kỳ 2 giây.

Các giá trị được đưa vào Queue để Task truyền dữ liệu tiếp nhận và gửi lên MQTT Broker.

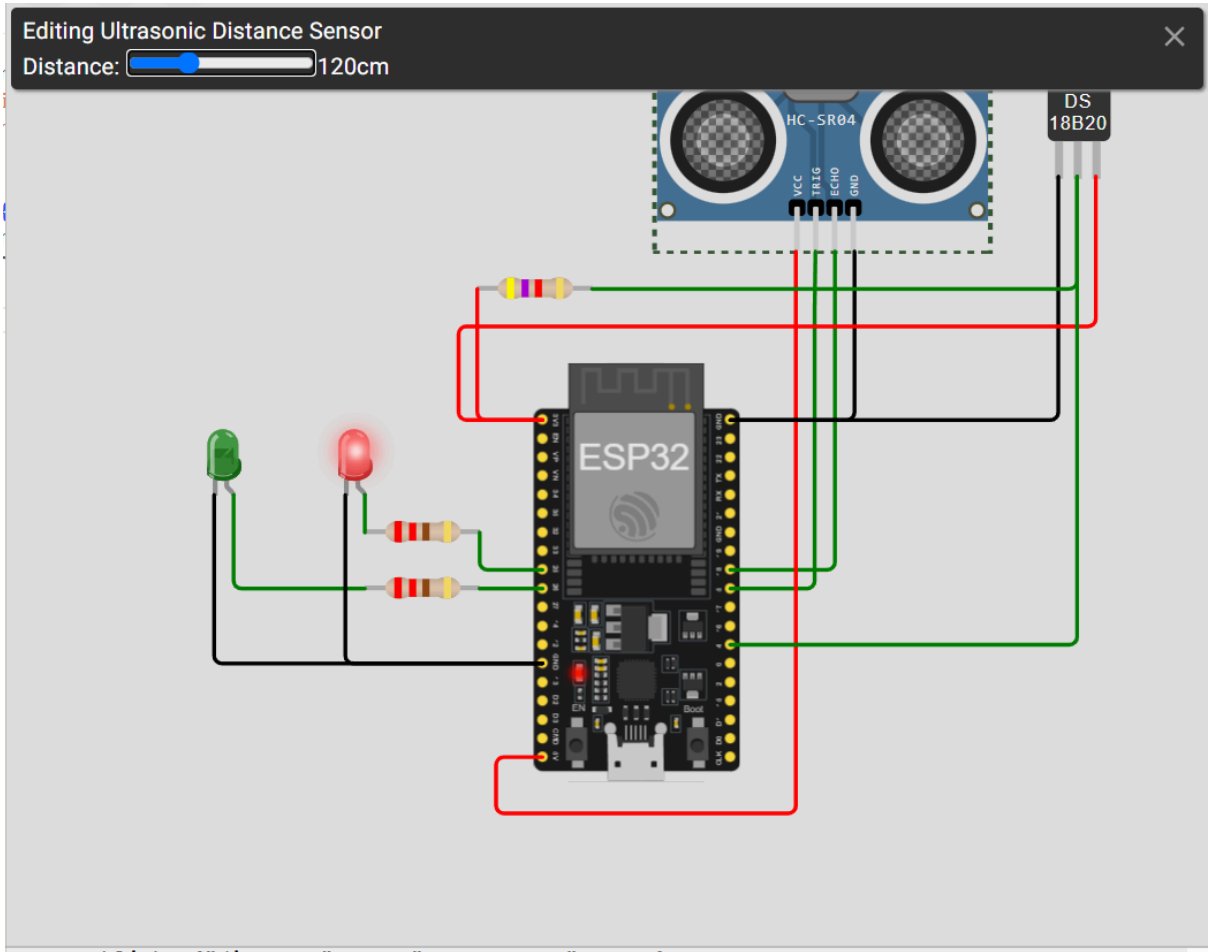
```
He thong da khoi tao!
Dang ket noi WiFi...
===== DU LIEU CAM BIEN =====
Khoang cach: 0.90 m
Nhiet do: 51.00 *C
WiFi connected!
10.10.0.2
MQTT connected!
MQTT State: {"mode":"AUTO","pump":false,"pumpDuty":0,"valve":false}
MQTT Publish: {"distance":0.90,"temperature":51.00}
MQTT Publish: {"distance":0.90,"temperature":51.00}
===== DU LIEU CAM BIEN =====
Khoang cach: 0.90 m
Nhiet do: 51.00 *C
MQTT Publish: {"distance":0.90,"temperature":51.00}
```

Hình 7.2 – Kết quả hiển thị dữ liệu trên Serial Monitor

7.3. Kết quả điều khiển tự động

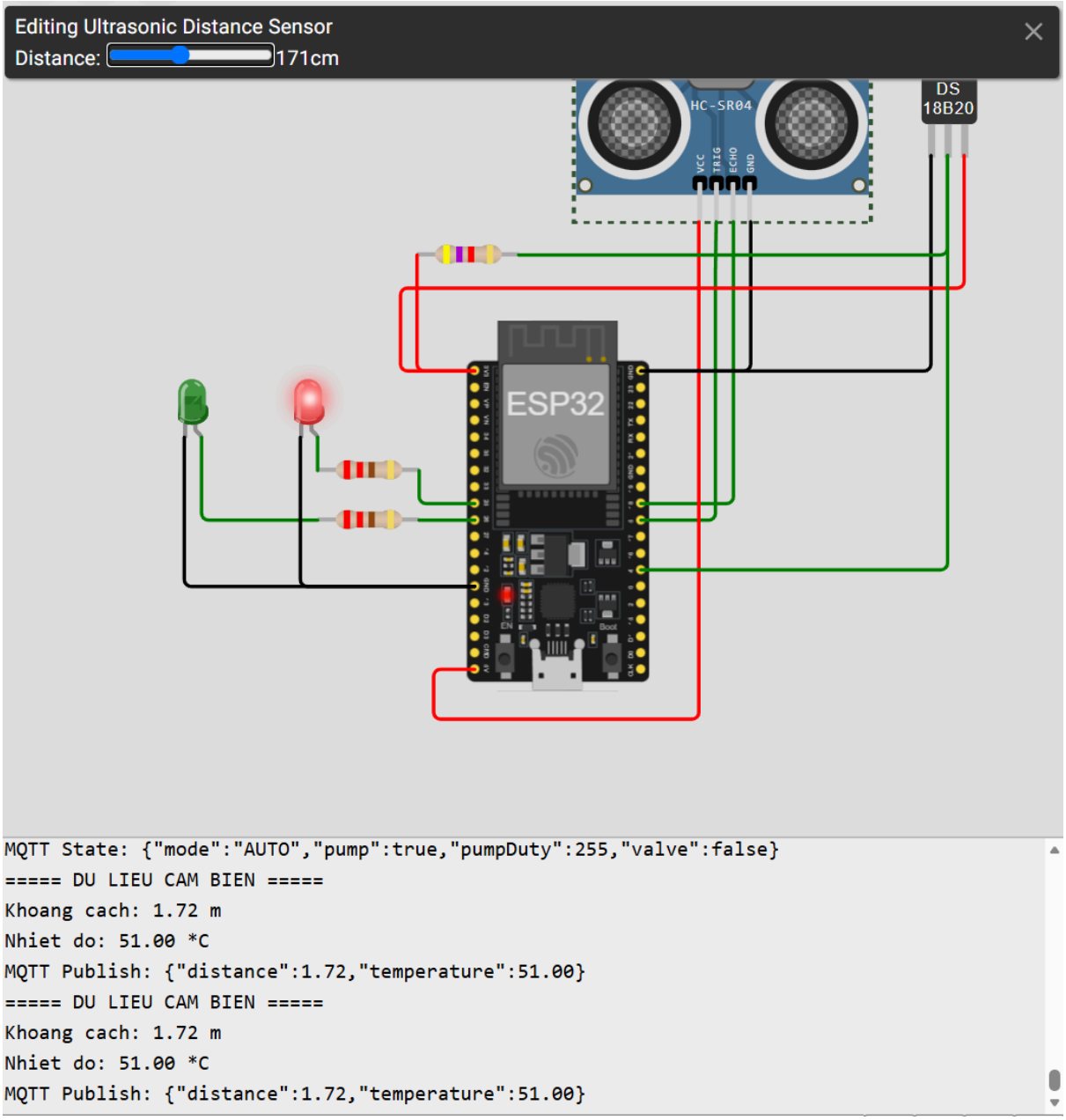
Hệ thống được kiểm tra với ba trường hợp:

Trường hợp 1: Khoảng cách từ 1 m đến dưới 1,5 m. Bơm hoạt động ở mức 50% trong 10 giây.



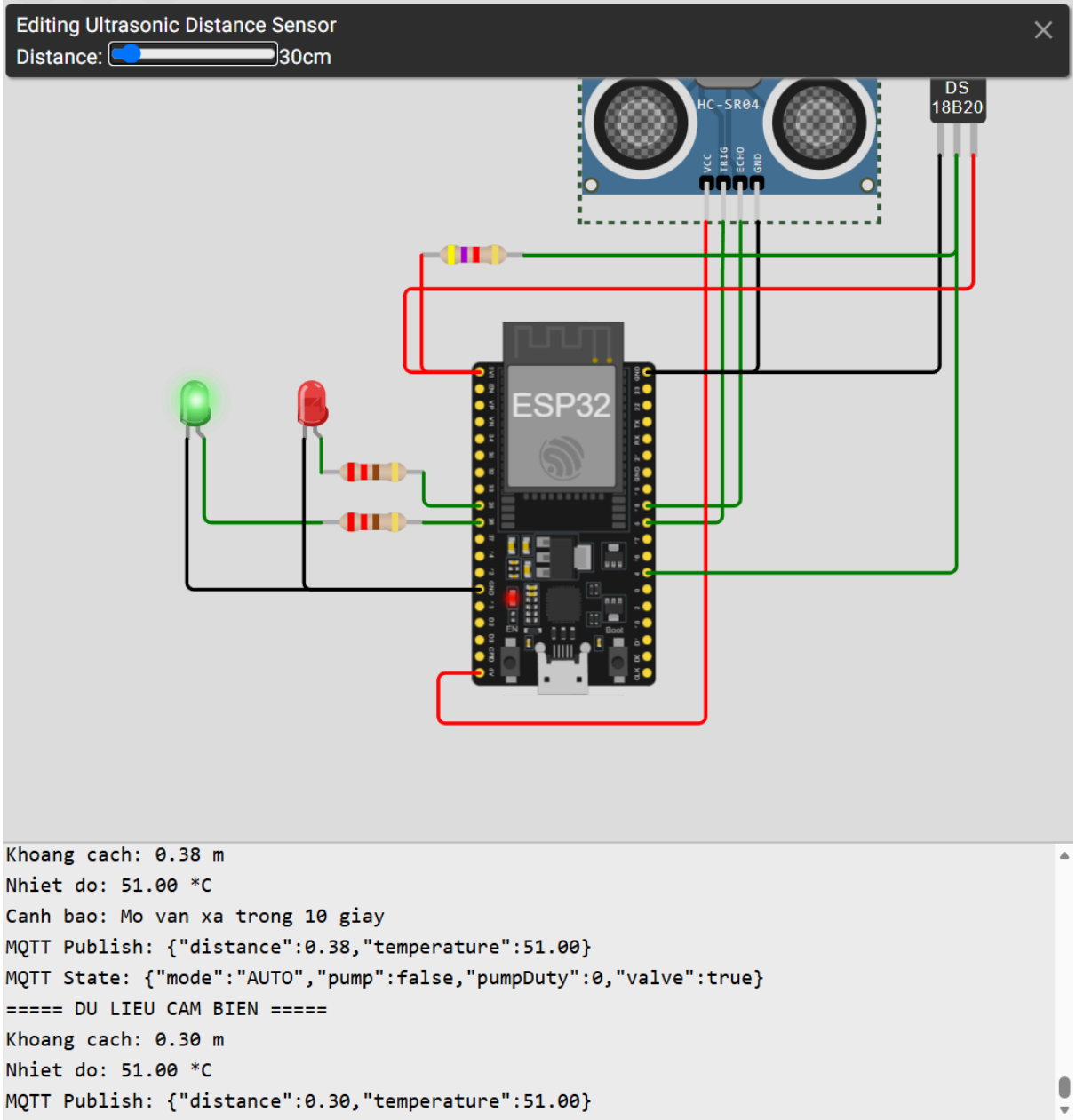
Hình 7.3 – Bơm hoạt động ở mức 50%

Trường hợp 2: Khoảng cách từ 1,5 m đến 2 m. Bơm hoạt động ở mức 100% trong 20 giây.



Hình 7.4 – Bơm hoạt động ở mức 100%

Trường hợp 3: Khoảng cách nhỏ hơn 0,5 m. Van xả được kích hoạt trong 10 giây.



Hình 7.5 – Van xả hoạt động

Kết quả mô phỏng cho thấy ESP32 có thể xác định các vùng khoảng cách và điều khiển thiết bị theo điều kiện đã thiết lập.

7.4. Kết quả điều khiển trên Website

Website nhận dữ liệu từ MQTT Broker và hiển thị thông số cảm biến theo thời gian thực.

Ở chế độ AUTO, hệ thống tự động điều khiển thiết bị theo ngưỡng. Ở chế độ MANUAL, người dùng có thể bật/tắt bơm và van xả trực tiếp trên giao diện.

Trạng thái thiết bị được cập nhật sau khi ESP32 xử lý lệnh.



Hình 7.6 – Giao diện Website ở chế độ AUTO



Hình 7.7 – Giao diện Website ở chế độ MANUAL

7.5. Đánh giá kết quả

Sau quá trình thực hiện, hệ thống đã đáp ứng các yêu cầu chính của đề tài:

- Đọc được dữ liệu khoảng cách và nhiệt độ từ hai cảm biến.
- Triển khai ba Task FreeRTOS trên ESP32.
- Sử dụng Queue để trao đổi dữ liệu giữa các Task.
- Kết nối Wi-Fi và truyền nhận dữ liệu bằng MQTT.
- Hiển thị thông số và biểu đồ trên Website.
- Thực hiện được chế độ điều khiển AUTO và MANUAL.
- Điều khiển được bơm và van xả theo các điều kiện đã thiết lập.

Hệ thống hiện được kiểm tra trong môi trường mô phỏng Wokwi. Hai LED chỉ mô phỏng trạng thái của bơm và van xả, chưa thực hiện điều khiển thiết bị công suất thực tế.